Übungsskript: Modulare Prozessdatenanalyse & OOP mit Python

Modul: Coding und Digitalisierung (Vorlesung 003) – Funktionen, Fehlerbehandlung, NumPy, Pandas-Zeitreihen und Objektorientierung

Wintersemester 2026, P. Kern

Inhaltsverzeichnis

1 Einführung und organisatorische Hinweise **4**

1.1 Verwendung von KI 4

1.2 Das Projekt: Prozessüberwachung „Klärwerk Nord“ (Belebungsbecken 1) . . 6

1.3 Lernziele 6

2 Modul 1: Projekt-Setup, Testdaten und das Copy-Paste-Problem **8**

2.1 Aufgabe 1.1: Ordnerstruktur und virtuelle Umgebung (venv) einrichten . . . 8

2.2 Aufgabe 1.2: Prozessdatensatz `bb12024.csv` anlegen 8

2.3 Aufgabe 1.3: Fehleranalyse im historischen „Spaghetti-Code“ (WET vs. DRY) . 9

2.4 Kontrollfragen zu Modul 1 10

3 Modul 2: Clean Code mit eigenen Funktionen (`def` & `return`) **11**

3.1 Aufgabe 2.1: Die erste eigene Funktion `berechne_mittelwert(liste)` . . 11

3.2 Aufgabe 2.2: Default-Parameter und Keyword-Argumente 11

3.3 Aufgabe 2.3: Mehrere Rückgabewerte als Tupel (Tuple Unpacking) 12

3.4 Aufgabe 2.4: Das Scope-Experiment (Lokaler vs. Globaler Gültigkeitsbereich) 12

3.5 Kontrollfragen zu Modul 2 13

4 Modul 3: Robuste Fehlerbehandlung mit `try` und `except` **14**

4.1 Aufgabe 3.1: Den Programmabsturz (`ValueError`) provozieren 14

4.2 Aufgabe 3.2: Gezieltes Abfangen mit `try` / `except ValueError` 14

4.3 Aufgabe 3.3: Diagnose-Parser mit Fehlerprotokollierung 14

4.4 Aufgabe 3.4: Schutz vor `ZeroDivisionError` und `TypeError` 15

4.5 Kontrollfragen zu Modul 3 16

5 Modul 4: Vektorisierte Signalverarbeitung mit NumPy **17**

5.1 Aufgabe 4.1: Vom bereinigten Sensor-Stream zum `numpy.ndarray` 17

5.2 Aufgabe 4.2: Vektorisierte Regelabweichung und Sättigungsberechnung . . . 17

5.3 Aufgabe 4.3: Kombinierte Boolesche Maskierung in NumPy 18
5.4 Kontrollfragen zu Modul 4 18

6 Modul 5: Einstieg in Pandas – DataFrames und Datenimport 20
6.1 Aufgabe 5.1: Einen ersten DataFrame aus einem Dictionary erzeugen 20
6.2 Aufgabe 5.2: Den Prozessdatensatz `bb12024.csv` laden 20
6.3 Aufgabe 5.3: Sichtprüfung mit `head()`, `tail()` und Strukturdiagnose mit `info()` 21
6.4 Aufgabe 5.4: Statistische Kurzanalyse mit `df.describe()` 21
6.5 Kontrollfragen zu Modul 5 22

7 Modul 6: Spaltenzugriff, Boolean Indexing und Prozess-Alarme 23
7.1 Aufgabe 6.1: Spalten selektieren und Einzelkennzahlen berechnen 23
7.2 Aufgabe 6.2: Kritische Sauerstoff-Zustände filtern (Boolean Indexing) 23
7.3 Aufgabe 6.3: Kombinierte Filterabfragen und neue berechnete Spalten 23
7.4 Kontrollfragen zu Modul 6 24

8 Modul 7: Datenbereinigung – NaN-Lücken und Plausibilitätsfilter 25
8.1 Aufgabe 7.1: Systematische Diagnose fehlender Werte (`isna()`) 25
8.2 Aufgabe 7.2: Experiment – Warum `fillna(0.0)` gefährlich sein kann 25
8.3 Aufgabe 7.3: Die dreistufige industrielle Bereinigungs-Pipeline 26
8.4 Kontrollfragen zu Modul 7 26

9 Modul 8: Zeitreihen-Analyse und Synchronisation (Resampling) 27
9.1 Aufgabe 8.1: Konvertierung mit `pd.to_datetime()` 27
9.2 Aufgabe 8.2: Den `DatetimeIndex` setzen und Zeitfenster ausschneiden . . . 27
9.3 Aufgabe 8.3: Synchronisation auf ein 5-Minuten-Raster (`resample('5min')`) 28
9.4 Aufgabe 8.4: Kapselung der Pipeline in einer Funktion 28
9.5 Kontrollfragen zu Modul 8 29

10 Modul 9: Objektorientierte Programmierung (OOP) – Klasse `Sensor` 30
10.1 Aufgabe 9.1: Klassendesign und Konstruktor `__init__` 30
10.2 Aufgabe 9.2: Methode `aktualisiere_wert(self, roh_input)` mit Schutzlogik 30
10.3 Aufgabe 9.3: Vektorisierte Auswertungsmethoden im Objekt 31
10.4 Kontrollfragen zu Modul 9 31

11 Modul 10: Große Abschluss-Synthese – Das System `BeckenAnalyse` 33
11.1 Aufgabe 10.1: Architektur der Klasse `BeckenAnalyse` 33

11.2 Aufgabe 10.2: Fehlertolerante Methode `aufbereiten_5min(self)` 33
11.3 Aufgabe 10.3: Alarm-Detektion und Management-Report 34
11.4 Aufgabe 10.4: Das Hauptprogramm ausführen und testen 34
11.5 Kontrollfragen zu Modul 10 35

1 Einführung und organisatorische Hinweise

Willkommen zum zweiten großen Übungsskript des Moduls „Coding und Digitalisierung“. Nachdem Sie im ersten Übungsblatt die Grundlagen von Variablen, Verzweigungen, Schleifen und ersten Arrays kennengelernt haben, vollziehen wir in diesem Arbeitsblatt den entscheidenden Schritt vom einfachen Einzelskript hin zu einer Software-Architektur.

Das vorliegende Dokument ist als zusammenhängendes Ingenieurprojekt konzipiert, das Sie Schritt für Schritt über mehrere Stunden hinweg bearbeiten können. Jedes Modul baut logisch auf dem vorherigen auf: Sie beginnen mit der Strukturierung von wiederverwendbarem Code über eigene Funktionen (Clean Code), härten Ihre Datenverarbeitung mit `try-except` gegen reale Sensorausfälle ab, verknüpfen vektorisierte Berechnungen in NumPy mit tabellarischen Prozessdaten in Pandas, synchronisieren asynchrone Zeitreihen auf ein exaktes 5-Minuten-Raster und überführen das gesamte System schließlich in eine saubere objektorientierte Programmarchitektur (OOP).

1.1 Verwendung von KI

Sie können die KI der Technischen Hochschule Köln kostenfrei für Ihre Fragen nutzen:

`https://chat.kiconnect.nrw/app`

> **Wichtig:** Fragen Sie nicht nach der reinen Komplettlösung einer Aufgabe, sondern immer nach einer ausführlichen Erklärung eines Konzepts oder einer Fehlermeldung. Eine bloße Lösung unterstützt das Lernen kaum – erst die nachvollziehbare Erklärung (z. B. *warum* ein `ValueError` auftritt oder welche Rolle `self` in einer Klasse spielt) fördert das Verständnis für Ihr Semesterprojekt.

Beispiel aus Vorlesung 003:

Frage: Was ist in Python der Unterschied zwischen `print()` und `return` am Ende einer Funktion, und warum liefert meine Variable den Wert None?

Antwort:

In Python haben `print()` und `return` völlig unterschiedliche Aufgaben:

Unterschiede im Überblick

- **`print()`:** Gibt lediglich einen Text sichtbar auf dem Bildschirm (Terminal) aus, reicht den berechneten Wert aber **nicht** an das Hauptprogramm zurück.
- **`return`:** Beendet die Funktion und übergibt das berechnete Ergebnis als echten Datenwert an die Stelle im Code, an der die Funktion aufgerufen wurde.
- **Rückgabewert None:** Besitzt eine Funktion keine `return`-Anweisung, gibt Python im Hintergrund automatisch den Platzhalterwert None (Nichts) zurück. Wenn Sie mit dem Ergebnis weiterrechnen wollen, müssen Sie zwingend `return` verwenden.

Beispiel

```
def verdopple(x):
    return x * 2

ergebnis = verdopple(4.5)
print(f"Berechneter Wert: {ergebnis}")
# Berechneter Wert: 9.0
```

1.2 Das Projekt: Prozessüberwachung „Klärwerk Nord“ (Belebungsbecken 1)

Sie arbeiten als Verfahrens- und Dateningenieur für das kommunale „Klärwerk Nord“. Im Zentrum der biologischen Abwasserreinigung steht das **Belebungsbecken 1 (BB1)**. Dort bauen Mikroorganismen schädliches Ammonium (NH_4) ab, wofür sie gelösten Sauerstoff (O_2) benötigen, der über leistungsstarke Gebläse eingeblasen wird.

Im Betrieb der Anlage treten jedoch vier klassische Digitalisierungsprobleme auf:

1. **Wartungsfeindlicher Bestands-Code:** Bisherige Auswertungsskripte bestehen aus hunderten Zeilen kopierter `for`-Schleifen (Copy-Paste-Code), in denen sich bereits gefährliche Berechnungsfehler eingeschlichen haben.
2. **Gestörte serielle Sensor-Streams:** Durch Feuchtigkeit, EMV-Störungen und Reinigungszyklen liefern Sensoren über die Schnittstelle immer wieder Texte wie `'Error'`, `'TIMEOUT'` oder `'SENSOR_FAULT'` statt gültiger Kommazahlen, was bisherige Python-Skripte sofort zum Absturz bringt.
3. **Lückenhafte und fehlerhafte Langzeitdaten:** Im historischen Messprotokoll `bb12024.csv` fehlen einzelne Messwerte (`NaN`), und durch einen verschobenen Nullpunkt am Ammonium-Sensor sind physikalisch unmögliche negative Konzentrationen (`-0.50 mg/L`) enthalten.
4. **Asynchrone Zeitstempel:** Die Sensoren senden ihre Daten ereignisgesteuert zu unregelmäßigen Uhrzeiten (z. B. `08:01`, `08:04`, `08:07`, `08:09`). Für die Prozessregelung und das Reporting müssen alle Kanäle zwingend auf einen einheitlichen **5-Minuten-Zeittakt** (`08:00`, `08:05`, `08:10`) synchronisiert werden.

Ihre Aufgabe ist es, Schritt für Schritt eine fehlertolerante, modulare und schließlich objektorientierte Analyse-Pipeline in Python (`main.py`) zu entwickeln, die das Belebungsbecken zuverlässig überwacht.

1.3 Lernziele

- Umsetzung des **DRY-Prinzips** (*Don’t Repeat Yourself*) und Kapselung von Berechnungslogik in eigenen Funktionen (`def`, Parameter, Default-Werte, Tupel-Rückgaben, Docstrings).
- Sicherer Umgang mit dem **Gültigkeitsbereich (Scope)** lokaler und globaler Variablen.
- Gezieltes Abfangen von Laufzeitfehlern mittels **`try` und `except`** (`ValueError`, `ZeroDivisionError`, `FileNotFoundError`).
- Kombination von **NumPy-Arrays** (vektorisierte Regelabweichungen, mehrdimensionale Boolesche Masken) mit heterogenen Datentabellen.
- Professioneller Import und Diagnose von Prozessdaten mit **Pandas DataFrames** (`pd.read_csv()`, `head()`, `info()`, `describe()`, Boolean Indexing).
- Systematische **Datenbereinigung** von `NaN`-Lücken und unplausiblen Messwerten über `dropna()`, `ffill()` und `interpolate()`.
- Zeitreihen-Konvertierung mittels `pd.to_datetime()`, Setzen eines **`DatetimeIndex`** sowie Taktsynchronisation durch **`df.resample('5min').mean()`**.

- Entwurf und Implementierung eigener **Klassen und Objekte (OOP)** mit Konstruktor (`__init__`), `self`, gekapselten Attributen und Methoden.

2 Modul 1: Projekt-Setup, Testdaten und das Copy-Paste-Problem

Bevor wir die ersten Funktionen schreiben, richten wir eine saubere Projektstruktur ein, installieren die benötigten Bibliotheken in einer isolierten virtuellen Umgebung und legen den Prozessdatensatz `bb12024.csv` an.

2.1 Aufgabe 1.1: Ordnerstruktur und virtuelle Umgebung (venv) einrichten

Erstellen Sie auf Ihrem Computer einen neuen Projektordner mit dem Namen `KlaerwerkProjekt` und öffnen Sie diesen in Visual Studio Code (VS Code). Legen Sie darin eine Datei `main.py` sowie eine Datei `bb12024.csv` an.

Tipp zur Einrichtung eines venv und Paketinstallation:

- Öffnen Sie das integrierte Terminal in VS Code und erstellen Sie eine virtuelle Umgebung:
  Für Windows: `python -m venv .venv`
  Für Mac/Linux: `python3 -m venv .venv`
- Aktivieren Sie das Environment anschließend über das Terminal:
  Für Windows: `.\.venv\Scripts\activate`
  Für Mac/Linux: `source .venv/bin/activate`
- Installieren Sie in der aktivierten Umgebung (erkennbar am vorangestellten `(.venv)`) die beiden Pakete numpy und pandas:
  `pip install numpy pandas`

2.2 Aufgabe 1.2: Prozessdatensatz `bb12024.csv` anlegen

Kopieren Sie exakt die folgenden 21 Zeilen (1 Kopfzeile und 20 Messzeilen aus einer einstündigen Messkampagne am Belebungsbecken 1) in die Datei `bb12024.csv` und speichern Sie diese ab:

```
zeit,o2_ist,o2_soll,nh4_mg_l,temp_c
2024-01-01 08:01:00,2.14,2.0,1.85,11.4
2024-01-01 08:04:00,2.08,2.0,1.88,11.4
2024-01-01 08:07:00,,2.0,1.92,11.3
2024-01-01 08:09:00,1.95,2.0,1.95,11.3
2024-01-01 08:12:00,2.02,2.0,1.90,11.3
2024-01-01 08:16:00,1.88,2.0,-0.50,11.5
2024-01-01 08:19:00,1.76,2.0,2.10,11.5
2024-01-01 08:22:00,0.42,2.0,4.35,11.6
2024-01-01 08:24:00,0.38,2.0,4.60,11.6
2024-01-01 08:27:00,0.45,2.0,,11.7
2024-01-01 08:31:00,1.25,2.5,3.80,11.7
2024-01-01 08:34:00,1.65,2.5,3.20,11.8
2024-01-01 08:37:00,,2.5,2.75,11.8
2024-01-01 08:39:00,2.10,2.5,2.40,11.9
2024-01-01 08:42:00,2.35,2.5,2.15,11.9
2024-01-01 08:46:00,2.48,2.5,1.95,12.0
2024-01-01 08:49:00,2.52,2.5,,12.0
2024-01-01 08:52:00,2.18,2.0,1.80,12.1
```

```
2024-01-01 08:54:00,2.04,2.0,1.75,12.1
2024-01-01 08:58:00,1.98,2.0,1.72,12.1
```

Blick in die Rohdaten: Schauen Sie sich die Zeilen genau an. In Zeile `08:07:00` und `08:37:00` steht zwischen zwei Kommas nichts (`,,`) – dort fehlt der O_2-Messwert! Um `08:16:00` hat der Ammonium-Sensor den Wert `-0.50` geliefert, und zwischen `08:22:00` und `08:27:00` bricht der Sauerstoffgehalt unter `0.5 mg/L` ein.

2.3 Aufgabe 1.3: Fehleranalyse im historischen „Spaghetti-Code“ (WET vs. DRY)

Ein Werkstudent hat vor Ihnen folgendes Skript geschrieben, um die Sauerstoff-Mittelwerte von drei Belebungsbecken zu berechnen. Kopieren Sie diesen Code in Ihre `main.py` und führen Sie ihn aus:

```python
print("=== Modul 1: Analyse des Bestands-Codes ===")

becken_1 = [4.2, 4.5, 4.1, 4.4]
becken_2 = [1.8, 2.1, 1.9, 2.0, 2.2, 1.7]
becken_3 = [0.5, 0.4, 0.6, 0.5, 0.5]

# Mittelwert Becken 1
summe_1 = 0.0
for wert in becken_1:
    summe_1 += wert
mw_1 = summe_1 / len(becken_1)

# Mittelwert Becken 2 (per Copy & Paste erzeugt)
summe_2 = 0.0
for wert in becken_2:
    summe_2 += wert
mw_2 = summe_2 / len(becken_1)

# Mittelwert Becken 3 (per Copy & Paste erzeugt)
summe_3 = 0.0
for wert in becken_3:
    summe_3 += wert
mw_3 = summe_3 / len(becken_3)

print(f"Mittelwert Becken 1: {mw_1:.2f} mg/L")
print(f"Mittelwert Becken 2: {mw_2:.2f} mg/L")
print(f"Mittelwert Becken 3: {mw_3:.2f} mg/L")
```

Ihre Aufgaben:

1. Prüfen Sie die Konsolenausgabe für `Becken 2`. Alle Einzelwerte von `becken_2` liegen um $2{,}0$ mg/L, aber das Skript gibt `2.92 mg/L` aus!
2. Suchen Sie den Copy-Paste-Fehler im obigen Code, korrigieren Sie ihn und prüfen Sie das korrigierte Ergebnis.

Erwartetes Ergebnis im Terminal (vor und nach der Korrektur):

```
=== Modul 1: Analyse des Bestands-Codes ===
Vor Korrektur -> Mittelwert Becken 2: 2.92 mg/L (FALSCH!)
Nach Korrektur -> Mittelwert Becken 1: 4.30 mg/L
Nach Korrektur -> Mittelwert Becken 2: 1.95 mg/L
Nach Korrektur -> Mittelwert Becken 3: 0.50 mg/L
```

2.4 Kontrollfragen zu Modul 1

Beantworten Sie für sich folgende Fragen (schreiben Sie die Antworten als Kommentar # in Ihr Skript):

1. Wofür stehen die Abkürzungen **DRY** und **WET** in der Softwareentwicklung?
2. Welche vier konkreten Probleme entstehen, wenn man in einem wachsenden Projekt mit 50 Sensoren Berechnungslogik per Copy-and-Paste vervielfältigt?
3. Warum stürzt Python bei dem Fehler in `mw_2 = summe_2 / len(becken_1)` nicht mit einer Fehlermeldung ab, und warum sind solche „logischen Fehler“ (Bugs) in der Verfahrenstechnik besonders gefährlich?

3 Modul 2: Clean Code mit eigenen Funktionen (`def` & `return`)

Um das DRY-Prinzip konsequent umzusetzen, darf jede Berechnungslogik im gesamten Programm nur an einer einzigen Stelle existieren. In diesem Modul ersetzen wir die redundanten Schleifen durch modulare Funktionen.

3.1 Aufgabe 2.1: Die erste eigene Funktion `berechne_mittelwert(liste)`

Schreiben Sie eine Funktion `berechne_mittelwert(liste)`, die eine beliebige Zahlenliste entgegennimmt und deren arithmetisches Mittel auf 2 Nachkommastellen gerundet per `return` zurückgibt.

- Fügen Sie direkt unter der `def`-Zeile einen erklärenden **Docstring** in dreifachen Anführungszeichen `"""..."""` ein.
- **Sonderfall abfangen:** Prüfen Sie zu Beginn mit `if len(liste) == 0:`, ob die übergebene Liste leer ist. Falls ja, soll die Funktion sofort `0.0` zurückgeben (verhindert eine Division durch Null!).
- Berechnen Sie die Summe mit einer `for`-Schleife, teilen Sie durch `len(liste)` und geben Sie das mit `round(..., 2)` gerundete Ergebnis per `return` zurück.
- Testen Sie Ihre Funktion anschließend mit `becken_1`, `becken_2`, `becken_3` sowie einer leeren Liste `leeres_becken = []`.

Erwartetes Ergebnis im Terminal:

```
=== Modul 2: Eigene Funktionen ===
O2 Becken 1: 4.3 mg/L
O2 Becken 2: 1.95 mg/L
O2 Becken 3: 0.5 mg/L
Test leeres Becken: 0.0 mg/L
```

3.2 Aufgabe 2.2: Default-Parameter und Keyword-Argumente

Sensoren im Klärwerk liefern gelegentlich unplausible Ausreißer (z. B. negative Werte bei Kalibrierfehlern oder Werte wie `999.9` bei Kurzschlüssen). Schreiben Sie eine flexible Filterfunktion mit **Default-Parametern**:

```python
def filtere_ausreisser(daten, min_wert=0.0, max_wert=50.0):
    """Behält nur Messwerte, die zwischen min_wert und max_wert liegen.
       """
    # Ihr Code hier:
```

Ihre Aufgaben:

1. Vervollständigen Sie die Funktion so, dass sie eine neue Liste `bereinigt` zurückgibt, die nur Werte w mit min_wert $\leq w \leq$ max_wert enthält.
2. Testen Sie die Funktion mit der Rohdaten-Liste `rohdaten = [-0.5, 4.1, 4.3, 999.9, 4.0, 85.2, 4.2, -12.0]` auf zwei Arten:

- **Aufruf 1 (Standardwerte nutzen):** Übergeben Sie nur `rohdaten`.
- **Aufruf 2 (Strenge O_2-Grenzen per Keyword-Argument):** Rufen Sie die Funktion mit `max_wert=10.0` auf.

Erwartetes Ergebnis im Terminal:

```
Standard-Filter (0.0 bis 50.0): [4.1, 4.3, 4.0, 4.2]
Strenger O2-Filter (0.0 bis 10.0): [4.1, 4.3, 4.0, 4.2]
```

3.3 Aufgabe 2.3: Mehrere Rückgabewerte als Tupel (Tuple Unpacking)

Oft reicht der reine Mittelwert für die Anlagensteuerung nicht aus; man benötigt gleichzeitig Minimum, Maximum und die Spannweite (Max − Min) einer Messreihe. In Python kann eine Funktion mehrere Werte durch Kommas getrennt zurückgeben.

1. Definieren Sie eine Funktion `berechne_kennzahlen(daten)`, die für eine übergebene Zahlenliste vier Werte berechnet: `mw` (gerundet auf 2 Stellen), `minimum`, `maximum` und `spannweite` (gerundet auf 2 Stellen).
2. Geben Sie alle vier Größen in einer einzigen Zeile zurück: `return mw, minimum, maximum, spannweite`. (Falls die Liste leer ist, geben Sie `return 0.0, 0.0, 0.0, 0.0` zurück.)
3. Wenden Sie zuerst `filtere_ausreisser` auf `rohdaten` an und übergeben Sie die bereinigte Liste an `berechne_kennzahlen`. Entpacken Sie das zurückgegebene Tupel direkt in vier Variablen:

```
sauber = filtere_ausreisser(rohdaten, max_wert=10.0)
mw, min_w, max_w, spanne = berechne_kennzahlen(sauber)
print(f"Kennzahlen -> MW: {mw} | Min: {min_w} | Max: {max_w} | Spanne: {
    spanne}")
```

Erwartetes Ergebnis im Terminal:

```
Kennzahlen -> MW: 4.15 | Min: 4.0 | Max: 4.3 | Spanne: 0.3
```

3.4 Aufgabe 2.4: Das Scope-Experiment (Lokaler vs. Globaler Gültigkeitsbereich)

In Ihrer Funktion `berechne_mittelwert(liste)` haben Sie intern eine Hilfsvariable `summe = 0.0` angelegt.

1. Versuchen Sie im Hauptprogramm (außerhalb der Funktion), direkt auf diese Variable zuzugreifen: `print("Interne Summe:", summe)`.
2. Führen Sie das Skript aus, beobachten Sie den entstehenden `NameError` im Terminal und kommentieren Sie die fehlerhafte Zeile anschließend wieder mit # aus, damit Ihr Skript weiterläuft.

3.5 Kontrollfragen zu Modul 2

1. Warum müssen Parameter *ohne* Standardwert im Funktionskopf (`def`) immer *vor* Parametern *mit* Default-Wert stehen?
2. Erklären Sie das „Black-Box-Prinzip“ lokaler Variablen: Warum ist es ein großer Vorteil für die Teamarbeit, dass die Variable `summe` nach dem `return` aus dem Speicher gelöscht wird?
3. Welchen Datentyp erzeugt Python automatisch, wenn Sie hinter `return` mehrere Werte mit Komma trennen?

4 Modul 3: Robuste Fehlerbehandlung mit `try` und `except`

Reale Felddaten aus industriellen Schnittstellen (RS485, Modbus, OPC UA) sind niemals perfekt. In diesem Modul sorgen wir dafür, dass Störungsmeldungen im Datenstrom unser Analyseprogramm nicht mehr zum Absturz bringen.

4.1 Aufgabe 3.1: Den Programmabsturz (`ValueError`) provozieren

Von der seriellen Schnittstelle des Sauerstoffsensors in Becken 1 erhalten Sie folgenden Rohdaten-Stream als Liste von Text-Strings:

```
sensor_stream = ['4.2', '4.5', 'Error', '4.1', 'TIMEOUT', '---', '4.4', '
    4.3']
```

Schreiben Sie testweise eine einfache `for`-Schleife ohne Schutzmechanismus, die versucht, jeden Eintrag mit `float(eintrag)` in eine Kommazahl umzuwandeln. Beobachten Sie den Absturz beim dritten Element (`'Error'`) und kommentieren Sie die ungeschützte Schleife danach aus.

4.2 Aufgabe 3.2: Gezieltes Abfangen mit `try` / `except ValueError`

Schreiben Sie eine Funktion `berechne_sicheren_mittelwert(roh_liste)`, die den gestörten `sensor_stream` sicher auswertet:

1. Legen Sie in der Funktion eine leere Liste `saubere_werte = []` an.
2. Iterieren Sie über `roh_liste`. Versuchen Sie im `try`-Block die Typumwandlung `wert = float(eintrag)` und hängen Sie den Wert an `saubere_werte` an.
3. Fangen Sie im `except ValueError:`-Block nicht-konvertierbare Strings ab und geben Sie eine Warnmeldung auf der Konsole aus: `print(f"Warnung: Ignoriere ungültigen Sensorwert '{eintrag}'")`.
4. Berechnen Sie am Ende den gerundeten Mittelwert der gültigen Zahlen (inklusive Schutz für den Fall, dass `len(saubere_werte) == 0` ist) und geben Sie ihn per `return` zurück.

Erwartetes Ergebnis im Terminal:

```
=== Modul 3: Fehlerbehandlung mit try/except ===
Warnung: Ignoriere ungültigen Sensorwert 'Error'
Warnung: Ignoriere ungültigen Sensorwert 'TIMEOUT'
Warnung: Ignoriere ungültigen Sensorwert '---'
Sicherer Mittelwert trotz 3 Störungen: 4.3 mg/L
```

4.3 Aufgabe 3.3: Diagnose-Parser mit Fehlerprotokollierung

In der Praxis möchte der Betriebsleiter nicht nur den bereinigten Mittelwert sehen, sondern auch wissen, *wie oft* und *mit welchen Fehlercodes* ein Sensor ausgefallen ist.

Entwickeln Sie eine erweiterte Funktion `parse_sensor_stream(roh_liste, min_wert=0.0, max_wert=20.0)`, die sowohl Textfehler (`ValueError`) als auch physikalische Ausreißer erkennt:

1. Erstellen Sie in der Funktion zwei Listen: `gueltige_werte = []` und `fehler_protokoll = []`.
2. Versuchen Sie per `try`, jeden Eintrag in `float` umzuwandeln:
   - Gelingt die Umwandlung, prüfen Sie zusätzlich mit einer `if`-Abfrage, ob der Zahlenwert im plausiblen Bereich min_wert $\leq$ wert $\leq$ max_wert liegt. Falls ja, kommt er in `gueltige_werte`. Falls nein (z. B. `'-99.9'` oder `'999.0'`), hängen Sie den String `f"Ausreisser: {wert}"` an `fehler_protokoll` an.
   - Schlägt die Umwandlung mit einem `ValueError` fehl, hängen Sie `f"Typfehler: {eintrag}"` an `fehler_protokoll` an.
3. Geben Sie beide Listen als Tupel zurück: `return gueltige_werte, fehler_protokoll`.
4. Testen Sie die Funktion mit folgendem Stream:

```
roh_stream_2 = ['2.1', '2.3', 'Error', '-99.9', '2.0', 'SENSOR_FAULT', '
    999.0', '2.2']
daten_ok, fehler_log = parse_sensor_stream(roh_stream_2)
print("Gültige Messwerte:", daten_ok)
print(f"Anzahl erkannter Fehler ({len(fehler_log)}):", fehler_log)
```

Erwartetes Ergebnis im Terminal:

```
Gültige Messwerte: [2.1, 2.3, 2.0, 2.2]
Anzahl erkannter Fehler (4): ['Typfehler: Error', 'Ausreisser: -99.9
    ', 'Typfehler: SENSOR_FAULT', 'Ausreisser: 999.0']
```

4.4 Aufgabe 3.4: Schutz vor `ZeroDivisionError` und `TypeError`

Bei der Berechnung des spezifischen Belüftungswirkungsgrads $\eta = \frac{\text{O}_2\text{-Eintrag}}{\text{Gebläseleistung}}$ kann die Gebläseleistung im Stillstand `0.0` betragen oder ein Parameter versehentlich als Text übergeben werden.

Schreiben Sie eine Funktion `berechne_wirkungsgrad(o2_eintrag, leistung_kw)`, die im `try`-Block `round(o2_eintrag / leistung_kw, 3)` berechnet und über zwei getrennte `except`-Blöcke abgesichert ist:

- `except ZeroDivisionError`: Gibt `"Hinweis: Gebläse im Stillstand (0 kW)."` aus und liefert `0.0` zurück.
- `except TypeError`: Gibt `"Fehler: Ungültiger Datentyp bei Wirkungsgradberechnung."` aus und liefert `0.0` zurück.

Testen Sie die Funktion mit `(15.0, 5.0)`, `(0.0, 0.0)` und `(15.0, "Aus")`.

4.5 Kontrollfragen zu Modul 3

1. Warum sollte man in Python immer die konkrete Fehlerart (z. B. `except ValueError:`) angeben, anstatt ein „nacktes“ `except:` ohne Fehlername zu verwenden?
2. Was passiert mit den restlichen Zeilen innerhalb eines `try`-Blocks, sobald in einer Zeile eine Exception ausgelöst wird?
3. Warum fängt `except ValueError:` einen physikalischen Ausreißer wie den Text `'-99.9'` *nicht* ab?

5 Modul 4: Vektorisierte Signalverarbeitung mit NumPy

Sobald unsere Rohdaten durch `parse_sensor_stream()` von Textfehlern befreit sind, liegen reine Zahlenlisten vor. Für schnelle mathematische Vektoroperationen (z. B. Regelabweichungen, Sättigungskurven und mehrdimensionale Schwellenwertmasken) überführen wir diese Listen in **NumPy-Arrays**.

5.1 Aufgabe 4.1: Vom bereinigten Sensor-Stream zum `numpy.ndarray`

Importieren Sie NumPy oben in Ihrem Skript mit `import numpy as np`. Gegeben sind drei gestörte Rohdaten-Streams (8 Messzeitpunkte) für den Sauerstoff-Istwert, den Sauerstoff-Sollwert und die Wassertemperatur:

```
stream_o2_ist  = ['2.10', '1.95', 'Error', '0.42', '0.38', '1.65', '2.10'
    , '2.35']
stream_o2_soll = ['2.00', '2.00', '2.00',  '2.00', '2.00', '2.50', '2.50'
    , '2.50']
stream_temp    = ['11.4', '11.3', '11.3',  '11.6', '11.6', '11.8', '11.9'
    , '11.9']
```

Da in `stream_o2_ist` an Position 2 ein `'Error'` steht, müssen alle drei Kanäle synchron bereinigt werden, damit die Array-Längen exakt übereinstimmen.

1. Schreiben Sie eine Funktion `synchronisiere_streams(s_ist, s_soll, s_temp)`, die mit einer Schleife über `range(len(s_ist))` läuft und in einem `try`-Block versucht, alle drei Werte am jeweiligen Index i per `float()` umzuwandeln.
2. Nur wenn *alle drei* Umwandlungen am Index i ohne `ValueError` gelingen, werden die drei Zahlen an drei lokale Listen angehängt.
3. Geben Sie die drei Listen direkt als NumPy-Arrays zurück: `return np.array(l_ist), np.array(l_soll), np.array(l_temp)`.

Erwartetes Ergebnis im Terminal:

```
=== Modul 4: Vektorisierte Analyse mit NumPy ===
Synchronisierte Array-Länge: 7 Messpunkte (1 defektes Tripel ü
    bersprungen)
O2-Ist Array:  [2.1  1.95 0.42 0.38 1.65 2.1  2.35]
O2-Soll Array: [2.  2.  2.  2.  2.5 2.5 2.5]
Temp Array:    [11.4 11.3 11.6 11.6 11.8 11.9 11.9]
```

5.2 Aufgabe 4.2: Vektorisierte Regelabweichung und Sättigungsberechnung

In der Regelungstechnik des Belebungsbeckens ist die **Regelabweichung** definiert als $e = \mathrm{O}_{2,\mathrm{Soll}} - \mathrm{O}_{2,\mathrm{Ist}}$. Ein positiver Wert bedeutet, dass Sauerstoff fehlt und das Gebläse stärker arbeiten muss. Zusätzlich hängt die maximale physikalische Sauerstoffsättigung C_s (in mg/L) näherungsweise nach folgender empirischer Formel von der Temperatur T (in °C) ab:

$$C_s(T) = 14{,}65 - 0{,}41 \cdot T + 0{,}008 \cdot T^2$$

Ihre Aufgaben (komplett ohne Schleifen!):

1. Berechnen Sie das Array `regelabweichung = o2_soll_np - o2_ist_np`.
2. Berechnen Sie das Array der Sauerstoffsättigung `c_saettigung = 14.65 - 0.41 * temp_np + 0.008 * (temp_np ** 2)`.
3. Berechnen Sie das Array der relativen Sauerstoffsättigung in Prozent: `saettigung_prozent = (o2_ist_np / c_saettigung) * 100`.
4. Geben Sie alle drei Ergebnis-Arrays mit `np.round(..., 2)` gerundet im Terminal aus.

Erwartetes Ergebnis im Terminal:

```
Regelabweichung (Soll - Ist): [-0.1   0.05  1.58  1.62  0.85  0.4
    0.15]
Max. O2-Sättigung Cs [mg/L]:  [11.02 11.04 10.97 10.97 10.93 10.9
    10.9 ]
Rel. O2-Sättigung [%]:        [19.06 17.66  3.83  3.46 15.1  19.26
    21.55]
```

5.3 Aufgabe 4.3: Kombinierte Boolesche Maskierung in NumPy

Ein Gebläse-Vollalarm soll ausgelöst werden, wenn die Regelabweichung größer als $0{,}8$ mg/L ist **und** gleichzeitig die Wassertemperatur mindestens $11{,}6\,^{\circ}$C beträgt (da Bakterien bei wärmerem Wasser schneller Sauerstoff verbrauchen).

1. Erstellen Sie eine kombinierte Boolesche Maske mit dem Operator &:

```
alarm_maske = (regelabweichung > 0.8) & (temp_np >= 11.6)
```

2. Filtern Sie das Array `o2_ist_np` mit dieser Maske und geben Sie die kritischen O_2-Werte sowie deren Anzahl (Tipp: `np.sum(alarm_maske)`) aus.
3. Berechnen Sie mit `np.mean()`, `np.std()`, `np.min()` und `np.max()` die statistischen Kennwerte des gesamten `o2_ist_np`-Arrays.

Erwartetes Ergebnis im Terminal:

```
Anzahl Gebläse-Vollalarme: 3
Kritische O2-Istwerte bei Alarm: [0.42 0.38 1.65]
NumPy Statistik O2-Ist -> MW: 1.56 | Std: 0.77 | Min: 0.38 | Max:
    2.35
```

5.4 Kontrollfragen zu Modul 4

1. Warum konnten wir in Aufgabe 4.1 nicht einfach `np.array(stream_o2_ist)` aufrufen und direkt damit rechnen? Welchen Datentyp (`dtype`) hätte NumPy wegen des Eintrags `'Error'` für das gesamte Array erzwungen?

2. Warum stößt ein reines NumPy-Array an seine Grenzen, sobald wir die Datei `bb12024.csv` inklusive der Spalte `zeit('2024-01-01 08:01:00')` und Spaltenüberschriften verarbeiten wollen?

3. Warum muss man bei der Verknüpfung zweier Bedingungen in NumPy (`maske1 & maske2`) zwingend runde Klammern um die einzelnen Vergleiche setzen?

6 Modul 5: Einstieg in Pandas – DataFrames und Datenimport

Wie in Vorlesung 003 gelernt, baut **Pandas** direkt auf der hohen C-Geschwindigkeit von NumPy auf, erweitert diese aber um benannte Spalten (`Columns`), einen intelligenten Zeilenindex (`Index`) und die Fähigkeit, unterschiedliche Datentypen (Zeitstempel, Text, Kommazahlen) in einer einzigen Tabelle (**DataFrame**) zu verwalten.

6.1 Aufgabe 5.1: Einen ersten DataFrame aus einem Dictionary erzeugen

Importieren Sie Pandas mit der weltweiten Standardkonvention `import pandas as pd`.

1. Erzeugen Sie aus folgendem Python-Dictionary einen ersten kleinen DataFrame `df_test` und geben Sie ihn auf der Konsole aus:

```
kurz_protokoll = {
    'zeit': ['08:00', '08:05', '08:10'],
    'becken': ['BB1', 'BB1', 'BB1'],
    'o2_mg_l': [4.12, 4.35, 3.98],
    'temp_c': [15.1, 15.2, 15.2]
}
df_test = pd.DataFrame(kurz_protokoll)
```

2. Prüfen Sie mit `type(df_test)` den Typ der gesamten Tabelle und mit `type(df_test['o2_mg_l'])` den Typ einer einzelnen Spalte.

Erwartetes Ergebnis im Terminal:

```
=== Modul 5: Pandas DataFrames & Datenimport ===
    zeit becken  o2_mg_l  temp_c
0  08:00    BB1     4.12    15.1
1  08:05    BB1     4.35    15.2
2  08:10    BB1     3.98    15.2
Typ der Tabelle: <class 'pandas.core.frame.DataFrame'>
Typ einer Spalte: <class 'pandas.core.series.Series'>
```

6.2 Aufgabe 5.2: Den Prozessdatensatz `bb12024.csv` laden

Nun laden wir unseren realen Prozessdatensatz `bb12024.csv` aus Modul 1 mit einer einzigen Codezeile in einen DataFrame `df`:

```
df = pd.read_csv('bb12024.csv')
zeilen, spalten = df.shape
print(f"Datensatz geladen! Zeilen: {zeilen} | Spalten: {spalten}")
print("Spaltennamen:", list(df.columns))
```

Praxis-Tipp für deutsche CSV-Exporte: Unsere Datei `bb12024.csv` nutzt das internationale Format (Komma als Spaltentrenner, Punkt als Dezimalzeichen). Sollten Sie im Semesterprojekt CSV-Dateien aus einem deutschen Excel oder einer Siemens-SPS erhalten, lauten die Parameter: `pd.read_csv('datei.csv',`

```
sep=';', decimal=',').
```

6.3 Aufgabe 5.3: Sichtprüfung mit `head()`, `tail()` und Strukturdiagnose mit `info()`

Bevor man mit einem Datensatz rechnet, führt man immer eine Sicht- und Strukturprüfung durch.

1. Lassen Sie sich mit `print(df.head(5))` die ersten 5 Zeilen und mit `print(df.tail(3))` die letzten 3 Zeilen von `df` ausgeben.
2. Rufen Sie `df.info()` auf, um den „Röntgenblick“ in die Tabellenstruktur durchzuführen.

Erwartetes Ergebnis im Terminal (Auszug):

```
                  zeit  o2_ist  o2_soll  nh4_mg_l  temp_c
0  2024-01-01 08:01:00    2.14      2.0      1.85    11.4
1  2024-01-01 08:04:00    2.08      2.0      1.88    11.4
2  2024-01-01 08:07:00     NaN      2.0      1.92    11.3
3  2024-01-01 08:09:00    1.95      2.0      1.95    11.3
4  2024-01-01 08:12:00    2.02      2.0      1.90    11.3

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 5 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   zeit      20 non-null     object
 1   o2_ist    18 non-null     float64
 2   o2_soll   20 non-null     float64
 3   nh4_mg_l  18 non-null     float64
 4   temp_c    20 non-null     float64
```

6.4 Aufgabe 5.4: Statistische Kurzanalyse mit `df.describe()`

Rufen Sie `print(df.describe().round(2))` auf. Untersuchen Sie die ausgegebene Statistik-Tabelle genau und finden Sie den physikalisch unmöglichen Kalibrierfehler in der Spalte `nh4_mg_l`.

Erwartetes Ergebnis im Terminal:

```
        o2_ist  o2_soll  nh4_mg_l  temp_c
count   18.00    20.00     18.00   20.00
mean     1.75     2.18      2.32   11.70
std      0.69     0.24      1.15    0.29
min      0.38     2.00     -0.50   11.30
25%      1.70     2.00      1.86   11.48
50%      2.00     2.00      1.94   11.70
75%      2.13     2.50      2.66   11.92
max      2.52     2.50      4.60   12.10
```

6.5 Kontrollfragen zu Modul 5

1. Warum taucht die Spalte `zeit` in der Tabelle von `df.describe()` überhaupt nicht auf?
2. Woran erkennen Sie in der Ausgabe von `df.info()`, dass in den Spalten `o2_ist` und `nh4_mg_l` jeweils 2 Messwerte fehlen?
3. Warum ist der Median (`50%`-Perzentil in `describe()`) bei der Beurteilung von Prozessdaten oft aussagekräftiger als der Mittelwert (`mean`)?

7 Modul 6: Spaltenzugriff, Boolean Indexing und Prozess-Alarme

In diesem Modul nutzen wir Pandas, um gezielte Abfragen auf unserer Prozess-Tabelle zu formulieren und kritische Betriebszustände im Belebungsbecken herauszufiltern.

7.1 Aufgabe 6.1: Spalten selektieren und Einzelkennzahlen berechnen

1. Greifen Sie gezielt auf die Spalte `df['temp_c']` zu und berechnen Sie deren Mittelwert (`.mean()`), Minimum (`.min()`) und Maximum (`.max()`).
2. Erzeugen Sie einen Teil-DataFrame `df_o2_uebersicht`, der ausschließlich die drei Spalten `'zeit'`, `'o2_ist'` und `'o2_soll'` enthält (Tipp: doppelte eckige Klammern `df[['zeit', 'o2_ist', 'o2_soll']]`), und geben Sie dessen erste 4 Zeilen aus.

Erwartetes Ergebnis im Terminal:

```
=== Modul 6: Selektion & Boolean Indexing ===
Temperatur-Statistik -> Mittel: 11.70 °C | Min: 11.30 °C | Max:
    12.10 °C
                  zeit  o2_ist  o2_soll
0  2024-01-01 08:01:00    2.14      2.0
1  2024-01-01 08:04:00    2.08      2.0
2  2024-01-01 08:07:00     NaN      2.0
3  2024-01-01 08:09:00    1.95      2.0
```

7.2 Aufgabe 6.2: Kritische Sauerstoff-Zustände filtern (Boolean Indexing)

Sinkt der Sauerstoffgehalt im Belebungsbecken unter $0{,}5$ mg/L, droht das Absterben der nitrifizierenden Bakterien.

1. Filtern Sie aus `df` alle Zeilen heraus, in denen `o2_ist < 0.5` gilt, und speichern Sie das Ergebnis in `kritisch_o2_df`.
2. Geben Sie die Anzahl der kritischen Messpunkte mit `len(kritisch_o2_df)` sowie die gefilterte Tabelle im Terminal aus.

Erwartetes Ergebnis im Terminal:

```
Anzahl kritischer O2-Messpunkte (< 0.5 mg/L): 3
                  zeit  o2_ist  o2_soll  nh4_mg_l  temp_c
7  2024-01-01 08:22:00    0.42      2.0      4.35    11.6
8  2024-01-01 08:24:00    0.38      2.0      4.60    11.6
9  2024-01-01 08:27:00    0.45      2.0       NaN    11.7
```

7.3 Aufgabe 6.3: Kombinierte Filterabfragen und neue berechnete Spalten

1. Filtern Sie alle Zeilen heraus, in denen entweder ein Sauerstoffmangel (`o2_ist < 0.5`) **oder** (`|`) eine überhöhte Ammonium-Belastung (`nh4_mg_l > 3.5`) vorliegt. Speichern Sie diese Zeilen in `alarm_df` und geben Sie deren Anzahl aus.

2. Fügen Sie dem DataFrame `df` eine neue berechnete Spalte `'o2_defizit'` hinzu, die für jede Zeile die Differenz `df['o2_soll'] - df['o2_ist']` berechnet. Geben Sie das maximale O_2-Defizit mit `df['o2_defizit'].max()` aus.

Erwartetes Ergebnis im Terminal:

```
Anzahl Zeilen mit O2-Mangel ODER NH4-Spitze (> 3.5 mg/L): 4
Maximales O2-Defizit (Soll - Ist): 1.62 mg/L
```

7.4 Kontrollfragen zu Modul 6

1. Warum benötigt man beim Abruf mehrerer Spalten (`df[['zeit', 'o2_ist']]`) zwei Paare eckiger Klammern? Was bewirkt das innere, was das äußere Klammerpaar?
2. Wie verhält sich `df['o2_ist'].mean()` gegenüber den beiden NaN-Werten in der Spalte? Stürzt Pandas ab, liefert es NaN zurück oder ignoriert es die Lücken automatisch?
3. Welchen Operator verwendet man in Pandas für eine logische ODER-Verknüpfung zweier Spaltenbedingungen?

8 Modul 7: Datenbereinigung – NaN-Lücken und Plausibilitätsfilter

Für nachgelagerte Regelungsmodelle und Zeitreihen-Analysen müssen wir unseren Datensatz nun systematisch von unplausiblen Fehlmessungen (`nh4_mg_l < 0`) und Datenlücken (NaN) befreien.

8.1 Aufgabe 7.1: Systematische Diagnose fehlender Werte (`isna()`)

Laden Sie `bb12024.csv` frisch in einen DataFrame `df_roh = pd.read_csv('bb12024.csv')`.

1. Ermitteln Sie mit `df_roh.isna().sum()` die exakte Anzahl fehlender Werte pro Spalte.
2. Berechnen Sie zusätzlich den prozentualen Anteil der Datenlücken pro Spalte über die Formel `(df_roh.isna().sum() / len(df_roh)) * 100`.

Erwartetes Ergebnis im Terminal:

```
=== Modul 7: Datenbereinigung ===
Fehlende Werte (absolut):
zeit        0
o2_ist      2
o2_soll     0
nh4_mg_l    2
temp_c      0
dtype: int64

Fehlende Werte (in %):
zeit         0.0
o2_ist      10.0
o2_soll      0.0
nh4_mg_l    10.0
temp_c       0.0
dtype: float64
```

8.2 Aufgabe 7.2: Experiment – Warum `fillna(0.0)` gefährlich sein kann

Vergleichen Sie drei verschiedene Ansätze zum Umgang mit NaN-Werten am Beispiel der Spalte `nh4_mg_l` (ohne `df_roh` dauerhaft zu überschreiben):

1. Berechnen Sie den Mittelwert von `nh4_mg_l`, wenn Sie NaN durch `0.0` ersetzen: `df_roh['nh4_mg_l'].fillna(0.0).mean()`.
2. Berechnen Sie den Mittelwert, wenn Sie Zeilen mit NaN löschen: `df_roh.dropna()['nh4_mg_l']`
3. Berechnen Sie den Mittelwert nach linearer Interpolation: `df_roh['nh4_mg_l'].interpolate()`
4. Geben Sie alle drei Mittelwerte gerundet auf 2 Nachkommastellen aus und vergleichen Sie die Abweichung!

Erwartetes Ergebnis im Terminal:

```
Vergleich NH4-Mittelwert bei verschiedenen NaN-Strategien:
1. Mit fillna(0.0) (verfälscht!): 2.09 mg/L
2. Mit dropna() (Zeilenverlust):  2.32 mg/L
3. Mit interpolate() (linear):    2.41 mg/L
```

8.3 Aufgabe 7.3: Die dreistufige industrielle Bereinigungs-Pipeline

Wenden Sie nun die dreistufige Best-Practice-Bereinigung aus Vorlesung 003 auf eine Arbeitskopie `df_clean = df_roh.copy()` an:

1. **Schritt 1 – Physikalisch unmögliche Werte entfernen:** In Zeile 5 (`08:16:00`) beträgt nh4_mg_l genau `-0.50`. Damit wir keine Zeile verlieren, in der nh4_mg_l aktuell noch NaN ist, behalten wir alle Zeilen, in denen nh4_mg_l `>= 0.0` gilt *oder* in denen nh4_mg_l ein NaN aufweist:

```
df_clean = df_clean[(df_clean['nh4_mg_l'] >= 0.0) | (df_clean['
    nh4_mg_l'].isna())].copy()
```

2. **Schritt 2 – Kurze Sensor-Aussetzer schließen:**
   - Nutzen Sie für o2_ist das Halteglied (Forward Fill): `df_clean['o2_ist'] = df_clean['o2_i`
   - Nutzen Sie für nh4_mg_l die lineare Interpolation: `df_clean['nh4_mg_l'] = df_clean['nh4_mg_l'].interpolate()`
3. **Schritt 3 – Randbereinigung & Kontrolle:** Rufen Sie abschließend `df_clean = df_clean.dropn` auf und prüfen Sie mit `df_clean.isna().sum()` sowie `df_clean['nh4_mg_l'].min()`, ob der Datensatz nun zu 100 % lückenfrei und physikalisch plausibel ist.

Erwartetes Ergebnis im Terminal:

```
Verbleibende Zeilen nach Plausibilitätsfilter: 19
Summe aller NaN-Werte nach Bereinigung: 0
Neues Minimum von nh4_mg_l: 1.72 mg/L (Kalibrierfehler -0.50
    erfolgreich entfernt!)
```

8.4 Kontrollfragen zu Modul 7

1. Erklären Sie den technischen Unterschied zwischen **Forward Fill** (`.ffill()`) und **linearer Interpolation** (`.interpolate()`). Wann entspricht `ffill()` exakt dem Verhalten einer speicherprogrammierbaren Steuerung (SPS)?
2. Warum sollte man nach einem `.ffill()` sicherheitshalber immer noch ein abschließendes `.dropna()` aufrufen? Was passiert nämlich, wenn ausgerechnet in der allerersten Zeile (Index 0) ein NaN-Wert steht?
3. Warum ist es gefährlich, mehrstündige Sensorausfälle (z. B. 100 aufeinanderfolgende NaN-Werte) blind per `interpolate()` aufzufüllen?

9 Modul 8: Zeitreihen-Analyse und Synchronisation (Resampling)

Noch immer liegt die Spalte `zeit` als gewöhnlicher Text (`object`) vor, und die Messzeitpunkte sind unregelmäßig (`08:01`, `08:04`, `08:07`, `08:09`). In diesem Modul schalten wir die Zeitreihen-Intelligenz von Pandas frei und synchronisieren alle Sensoren auf ein exaktes **5-Minuten-Raster**.

9.1 Aufgabe 8.1: Konvertierung mit `pd.to_datetime()`

Arbeiten Sie mit Ihrem bereinigten DataFrame `df_clean` aus Modul 7 weiter.

1. Geben Sie den aktuellen Datentyp der Spalte `zeit` mit `df_clean['zeit'].dtype` aus.
2. Wandeln Sie die Text-Spalte in echte `datetime64[ns]`-Objekte um (inklusive `errors='coerce'` als Schutz gegen kaputte Zeit-Strings):

```
df_clean['zeit'] = pd.to_datetime(df_clean['zeit'], errors='coerce')
```

3. Geben Sie den neuen Datentyp, den Wochentag des ersten Messpunkts (`df_clean['zeit'].dt.d`) sowie die gesamte Zeitspanne der Messung (`df_clean['zeit'].max() - df_clean['zeit'].`) aus.

Erwartetes Ergebnis im Terminal:

```
=== Modul 8: Zeitreihen & 5-Minuten-Resampling ===
Datentyp 'zeit' vorher: object -> nachher: datetime64[ns]
Wochentag der Messung: Monday
Gesamtdauer des Messprotokolls: 0 days 00:57:00
```

9.2 Aufgabe 8.2: Den `DatetimeIndex` setzen und Zeitfenster ausschneiden

1. Machen Sie die Spalte `zeit` mit `df_clean.set_index('zeit', inplace=True)` zum neuen Index des DataFrames.
2. Schneiden Sie über `df_clean.loc['2024-01-01 08:20:00':'2024-01-01 08:35:00']` exakt das 15-minütige Störungsfenster aus und geben Sie die Spalten `['o2_ist', 'nh4_mg_l']` für diesen Zeitraum aus.

Erwartetes Ergebnis im Terminal:

```
Zeitfenster-Slicing (08:20 bis 08:35 Uhr):
                     o2_ist  nh4_mg_l
zeit
2024-01-01 08:22:00    0.42     4.350
2024-01-01 08:24:00    0.38     4.600
2024-01-01 08:27:00    0.45     4.200
2024-01-01 08:31:00    1.25     3.800
2024-01-01 08:34:00    1.65     3.200
```

9.3 Aufgabe 8.3: Synchronisation auf ein 5-Minuten-Raster (`resample('5min')`)

Jetzt takten wir die asynchronen Messwerte auf ein striktes 5-Minuten-Intervall:

1. Führen Sie das Resampling über den Mittelwert durch und schließen Sie eventuell entstehende leere 5-Minuten-Körbe per linearer Interpolation:

```
df_5min = df_clean.resample('5min').mean().interpolate()
```

2. Geben Sie die Anzahl der Zeilen von `df_5min` sowie die ersten 6 Zeilen gerundet auf 2 Nachkommastellen (`df_5min.head(6).round(2)`) aus.
3. Erstellen Sie zusätzlich ein 15-Minuten-Resampling `df_15min = df_clean.resample('15min')` und geben Sie dieses vollständig aus.

Erwartetes Ergebnis im Terminal:

```
Anzahl 5-Minuten-Intervalle: 12
                     o2_ist  o2_soll  nh4_mg_l  temp_c
zeit
2024-01-01 08:00:00    2.11      2.0      1.86   11.40
2024-01-01 08:05:00    2.02      2.0      1.94   11.30
2024-01-01 08:10:00    2.02      2.0      1.90   11.30
2024-01-01 08:15:00    1.76      2.0      2.10   11.50
2024-01-01 08:20:00    0.40      2.0      4.48   11.60
2024-01-01 08:25:00    0.45      2.0      4.20   11.70

15-Minuten-Übersicht (Viertelstunden-Mittelwerte):
                     o2_ist  o2_soll  nh4_mg_l  temp_c
zeit
2024-01-01 08:00:00    2.05     2.00      1.90   11.34
2024-01-01 08:15:00    0.75     2.00      3.81   11.60
2024-01-01 08:30:00    1.85     2.50      2.86   11.82
2024-01-01 08:45:00    2.24     2.20      1.92   12.06
```

9.4 Aufgabe 8.4: Kapselung der Pipeline in einer Funktion

Verbinden Sie Ihr Wissen aus Modul 2, 7 und 8: Schreiben Sie eine saubere, wiederverwendbare Funktion `lade_und_resample(dateipfad, intervall='5min')`, die:

1. die CSV-Datei einliest,
2. unplausible Zeilen mit `nh4_mg_l < 0` ausschließt (ohne NaN-Zeilen zu verlieren),
3. die Spalte `'zeit'` per `pd.to_datetime()` umwandelt und als Index setzt,
4. das Resampling mit dem übergebenen Parameter `intervall` (`.resample(intervall).mean()`) durchführt,
5. alle Lücken per `.interpolate().ffill().bfill()` schließt und den fertigen DataFrame per `return` zurückgibt.

9.5 Kontrollfragen zu Modul 8

1. Welche konkrete Fehlermeldung wirft Pandas, wenn Sie `df.resample('5min').mean()` aufrufen, aber vorher vergessen haben, `df.set_index('zeit')` auszuführen?
2. Welche beiden Einzelmessungen aus unserer Rohdatei `bb12024.csv` werden beim 5-Minuten-Resampling im Zeitkorb `2024-01-01 08:00:00` zusammengefasst, sodass der Wert `o2_ist = 2.11` entsteht?
3. Welchen angenehmen Nebeneffekt hat die Mittelwertbildung beim Resampling (`.mean()`) auf hochfrequentes Sensorrauschen?

10 Modul 9: Objektorientierte Programmierung (OOP) – Klasse Sensor

In großen Industrieanlagen mit hunderten Messstellen führt prozeduraler Code mit losen Variablen und Listen schnell ins Chaos. In diesem Modul modellieren wir einen realen Industriemessfühler als eigenständige **Klasse (`class Sensor`)**, die ihre eigenen Daten (Attribute) und ihre eigene Prüf- und Auswertungslogik (Methoden) in sich vereint.

10.1 Aufgabe 9.1: Klassendesign und Konstruktor `__init__`

Entwerfen Sie eine Klasse `Sensor` mit dem Konstruktor `__init__(self, sensor_id, typ, einheit, min_grenze=0.0, max_grenze=50.0)`:

- Legen Sie im Konstruktor folgende Objekt-Attribute über `self` an:
  - `self.id = sensor_id`
  - `self.typ = typ`
  - `self.einheit = einheit`
  - `self.min_grenze = min_grenze`
  - `self.max_grenze = max_grenze`
  - `self.aktueller_wert = None`
  - `self.historie = []` (leere Liste für gültige Messwerte)
  - `self.fehler_zaehler = 0` (Zähler für abgelehnte Messwerte)
- Erzeugen Sie testweise zwei konkrete Objekte (Instanzen):
  - `s_o2 = Sensor(101, 'Sauerstoff', 'mg/L', min_grenze=0.0, max_grenze=15.0)`
  - `s_nh4 = Sensor(102, 'Ammonium', 'mg/L', min_grenze=0.0, max_grenze=20.0)`

10.2 Aufgabe 9.2: Methode `aktualisiere_wert(self, roh_input)` mit Schutzlogik

Erweitern Sie Ihre Klasse `Sensor` um die Methode `aktualisiere_wert(self, roh_input)`:

1. Versuchen Sie in einem `try`-Block, `roh_input` mit `wert = float(roh_input)` in eine Kommazahl umzuwandeln.
2. Fangen Sie im `except (ValueError, TypeError):`-Block ungültige Eingaben (wie `'Error'` oder None) ab: Erhöhen Sie `self.fehler_zaehler += 1` und geben Sie aus: `print(f"[Sensor {self.id}] Typfehler ignoriert: {roh_input}")`.
3. Falls die Umwandlung in `float` geklappt hat, prüfen Sie, ob self.min_grenze $\leq$ wert $\leq$ self.max_grenze gilt:
   - **Wenn ja:** Setzen Sie `self.aktueller_wert = wert` und hängen Sie `wert` an `self.historie` an.
   - **Wenn nein:** Erhöhen Sie `self.fehler_zaehler += 1` und geben Sie aus: `print(f"[Sensor {self.id}] Grenzwert-Ausreißer ignoriert: {wert} {self.einheit}")`.

10.3 Aufgabe 9.3: Vektorisierte Auswertungsmethoden im Objekt

Fügen Sie der Klasse `Sensor` zwei weitere Methoden hinzu:

1. `berechne_statistik(self)`:
   - Prüft zuerst, ob `len(self.historie) == 0` ist (falls ja, Rückgabe `0.0, 0.0, 0.0, 0.0`).
   - Wandelt `self.historie` intern in ein NumPy-Array `arr = np.array(self.historie)` um.
   - Gibt Mittelwert, Standardabweichung, Minimum und Maximum (jeweils auf 2 Stellen gerundet) als Tupel zurück.
2. `zeige_status(self)`: Ruft intern `self.berechne_statistik()` auf und gibt eine übersichtliche Statuszeile mit Sensor-ID, Typ, Anzahl gültiger Messungen, Fehlerzähler, aktuellem Wert sowie Mittelwert und Standardabweichung auf der Konsole aus.

Testcode für Ihre Klasse `Sensor`:

```
test_daten_o2  = ['2.14', '2.08', 'Error', '1.95', '-5.0', '0.42', '99.9', '2.10']
test_daten_nh4 = ['1.85', '1.88', '-0.50', '4.35', 'TIMEOUT', '3.80', '2.40']

for eintrag in test_daten_o2:
    s_o2.aktualisiere_wert(eintrag)

for eintrag in test_daten_nh4:
    s_nh4.aktualisiere_wert(eintrag)

s_o2.zeige_status()
s_nh4.zeige_status()
```

Erwartetes Ergebnis im Terminal:

```
=== Modul 9: Objektorientierte Sensor-Modellierung ===
[Sensor 101] Typfehler ignoriert: Error
[Sensor 101] Grenzwert-Ausreißer ignoriert: -5.0 mg/L
[Sensor 101] Grenzwert-Ausreißer ignoriert: 99.9 mg/L
[Sensor 102] Grenzwert-Ausreißer ignoriert: -0.5 mg/L
[Sensor 102] Typfehler ignoriert: TIMEOUT
Sensor 101 (Sauerstoff) | Gültig: 5 | Fehler: 3 | Aktuell: 2.1 mg/L | MW: 1.74 +/- 0.66 mg/L
Sensor 102 (Ammonium)   | Gültig: 5 | Fehler: 2 | Aktuell: 2.4 mg/L | MW: 2.86 +/- 1.03 mg/L
```

10.4 Kontrollfragen zu Modul 9

1. Erklären Sie die vier Grundbegriffe der OOP (**Klasse**, **Objekt/Instanz**, **Attribut**, **Methode**) anhand Ihres `Sensor`-Beispiels.

2. Warum besitzen `s_o2` und `s_nh4` jeweils eine völlig eigene Liste `historie`, obwohl beide aus derselben Klasse `Sensor` erzeugt wurden? Welche Rolle spielt dabei das Schlüsselwort `self`?

3. Wann genau wird die Methode `__init__` ausgeführt? Muss man sie nach dem Erzeugen eines Objekts noch einmal manuell aufrufen?

11 Modul 10: Große Abschluss-Synthese – Das System BeckenAnalyse

Im letzten Modul führen wir alle Werkzeuge aus den vier Vorlesungsstunden (Funktionen, `try/except`, NumPy, Pandas-Zeitreihen und OOP) in einer übergeordneten Systemklasse **`BeckenAnalyse`** zusammen. Diese Klasse repräsentiert den digitalen Zwilling eines kompletten Belebungsbeckens und dient Ihnen als direkte Architekturvorlage für Ihr Semesterprojekt.

11.1 Aufgabe 10.1: Architektur der Klasse BeckenAnalyse

Erstellen Sie eine Klasse `BeckenAnalyse` mit folgendem Konstruktor:

```
class BeckenAnalyse:
    """Objektorientiertes Analyse- und Überwachungssystem für ein
       Belebungsbecken."""
    def __init__(self, becken_name, csv_pfad):
        self.name = becken_name
        self.pfad = csv_pfad
        self.roh_df = None
        self.df_5min = None
        self.ist_aufbereitet = False
```

11.2 Aufgabe 10.2: Fehlertolerante Methode `aufbereiten_5min(self)`

Implementieren Sie innerhalb der Klasse `BeckenAnalyse` die Methode `aufbereiten_5min(self)`:

1. **Datei-Import mit `try/except`:** Versuchen Sie im `try`-Block, `self.pfad` mit `pd.read_csv(self.p` in `self.roh_df` zu laden. Fangen Sie einen `FileNotFoundError` ab (falls ein falscher Dateiname übergeben wurde), geben Sie eine klare Fehlermeldung aus und beenden Sie die Methode mit `return False`.
2. **Plausibilitätsbereinigung:** Arbeiten Sie auf einer Kopie `df = self.roh_df.copy()`. Entfernen Sie negative Fehlmessungen in `nh4_mg_l` über:

   ```
   df = df[(df['nh4_mg_l'] >= 0.0) | (df['nh4_mg_l'].isna())].copy()
   ```

3. **Lücken schließen & Zeitindex setzen:**
   - Schließen Sie Lücken in `o2_ist` per `.ffill()` und in `nh4_mg_l` per `.interpolate()`.
   - Konvertieren Sie `df['zeit']` mit `pd.to_datetime()` und setzen Sie `zeit` als Index.
4. **5-Minuten-Resampling & berechnete Spalten:**
   - Erzeugen Sie `self.df_5min = df.resample('5min').mean().interpolate()`.
   - Fügen Sie `self.df_5min` eine neue Spalte `'o2_defizit'` hinzu (`o2_soll - o2_ist`).
   - Berechnen Sie zusätzlich eine Spalte `'geblaese_kw'` (geschätzte Gebläseleistung in kW) mithilfe von NumPy: Immer wenn `o2_defizit > 0` ist, beträgt die Leistung $15{,}0 + 25{,}0 \cdot$ o2_defizit, andernfalls im Grundlastbetrieb $15{,}0$ kW (Tipp: `np.where(self.df_5min['` `> 0, 15.0 + 25.0 * self.df_5min['o2_defizit'], 15.0)`).
5. Setzen Sie `self.ist_aufbereitet = True` und geben Sie `return True` zurück.

11.3 Aufgabe 10.3: Alarm-Detektion und Management-Report

Fügen Sie Ihrer Klasse `BeckenAnalyse` zwei Auswertungsmethoden hinzu:

1. `pruefe_grenzwerte(self, o2_min=0.5, nh4_max=4.0)`:
   - Prüft zuerst `if not self.ist_aufbereitet:`, gibt sonst einen Hinweis aus und bricht ab.
   - Filtert aus `self.df_5min` alle 5-Minuten-Intervalle heraus, in denen `o2_ist < o2_min` oder `nh4_mg_l > nh4_max` aufgetreten ist, und gibt diesen Alarm-DataFrame zurück.
2. `zeige_report(self)`:
   - Gibt einen sauber formatierten Management-Bericht auf der Konsole aus: Name des Beckens, Anzahl der Rohzeilen vs. Anzahl der 5-Minuten-Intervalle, Mittelwerte von `o2_ist`, `nh4_mg_l`, `temp_c` und `geblaese_kw` sowie die Anzahl der gefundenen Alarm-Intervalle aus `self.pruefe_grenzwerte()`.
   - Exportiert den fertig bereinigten und synchronisierten 5-Minuten-Datensatz über `self.df_5min.round(2).to_csv('bb12024_clean_5min.csv')` als neue CSV-Datei im Projektordner.

11.4 Aufgabe 10.4: Das Hauptprogramm ausführen und testen

Testen Sie Ihre fertige Architektur am Ende Ihrer `main.py` sowohl mit einem nicht existierenden Dateipfad (Test der Fehlerbehandlung) als auch mit Ihrer echten Datei `bb12024.csv`:

```
print("\n=== Modul 10: Objekt-Synthese BeckenAnalyse ===")

# 1. Negativtest: Falscher Dateipfad (darf NICHT abstürzen!)
becken_fehler = BeckenAnalyse("Testbecken (Falscher Pfad)", "existiert_nicht.csv")
becken_fehler.aufbereiten_5min()
becken_fehler.zeige_report()

# 2. Produktivlauf mit Belebungsbecken 1
bb1 = BeckenAnalyse("Belebungsbecken 1 (Klärwerk Nord)", "bb12024.csv")
if bb1.aufbereiten_5min():
    bb1.zeige_report()
    alarme = bb1.pruefe_grenzwerte(o2_min=0.5, nh4_max=4.0)
    print("\nDetektierte 5-Minuten-Alarmintervalle:")
    print(alarme[['o2_ist', 'nh4_mg_l', 'o2_defizit', 'geblaese_kw']].round(2))
```

Erwartetes Ergebnis im Terminal:

```
=== Modul 10: Objekt-Synthese BeckenAnalyse ===
[Fehler] Datei 'existiert_nicht.csv' wurde nicht gefunden!
[Hinweis] Report für 'Testbecken (Falscher Pfad)' nicht möglich: Daten nicht aufbereitet.

============================================================
PROZESS-REPORT: Belebungsbecken 1 (Klärwerk Nord) (5min-Takt)
```

```
============================================================
Rohdaten-Zeilen: 20 | Synchronisierte 5min-Intervalle: 12
Mittlerer O2-Gehalt:       1.76 mg/L (Soll-Mittel: 2.21 mg/L)
Mittlere NH4-Belastung:    2.62 mg/L
Mittlere Wassertemperatur: 11.71 °C
Mittlere Gebläseleistung:  27.49 kW
Detektierte Alarm-Takte:   2 Intervalle
Export abgeschlossen:      'bb12024_clean_5min.csv' gespeichert.
============================================================

Detektierte 5-Minuten-Alarmintervalle:
                     o2_ist  nh4_mg_l  o2_defizit  geblaese_kw
zeit
2024-01-01 08:20:00    0.40      4.48        1.60        55.00
2024-01-01 08:25:00    0.45      4.20        1.55        53.75
```

11.5 Kontrollfragen zu Modul 10

1. Wie viele Zeilen Code müssen Sie im Hauptprogramm ergänzen, wenn das Klärwerk morgen drei weitere Belebungsbecken (`bb22024.csv`, `bb32024.csv`, `bb42024.csv`) in Betrieb nimmt und exakt nach demselben Standard auswerten möchte?
2. Warum haben wir in `BeckenAnalyse` ein Attribut `self.ist_aufbereitet = False` eingeführt, das vor dem Ausführen von `zeige_report()` geprüft wird?
3. Reflektieren Sie Ihren Lernfortschritt über alle 10 Module: Wie greifen **eigene Funktionen**, **`try/except`**, **NumPy**, **Pandas-Zeitreihen** und **Klassen (OOP)** in dieser finalen Lösung ineinander, und welche Komponente übernimmt dabei welche Aufgabe?

Herzlichen Glückwunsch!

Sie haben das komplexe Übungsprojekt zur Vorlesung 003 erfolgreich abgeschlossen und eine industrietaugliche, objektorientierte Zeitreihen-Pipeline in Python implementiert.